PROGETTO

PROGRAMMING FOR DATA SCIENCE

Salvatore De Gregorio

OBIETTIVO

- **Assistente Scouting :**
- Creare un sistema con 2 server MCP interconnessi che rispondano usando i dati reali di FC 25, fornendo all’utente dati reali e grafici per un aiuto nella configurazione della propria squadra.

- **Il Limite degli LLM:** Modelli come ChatGPT/Claude non hanno accesso nativo ai dati

- **Risolvere il problema delle "Allucinazioni" dell'AI:**
- Le AI spesso inventano i numeri. Il mio sistema legge il database e dice la verità.

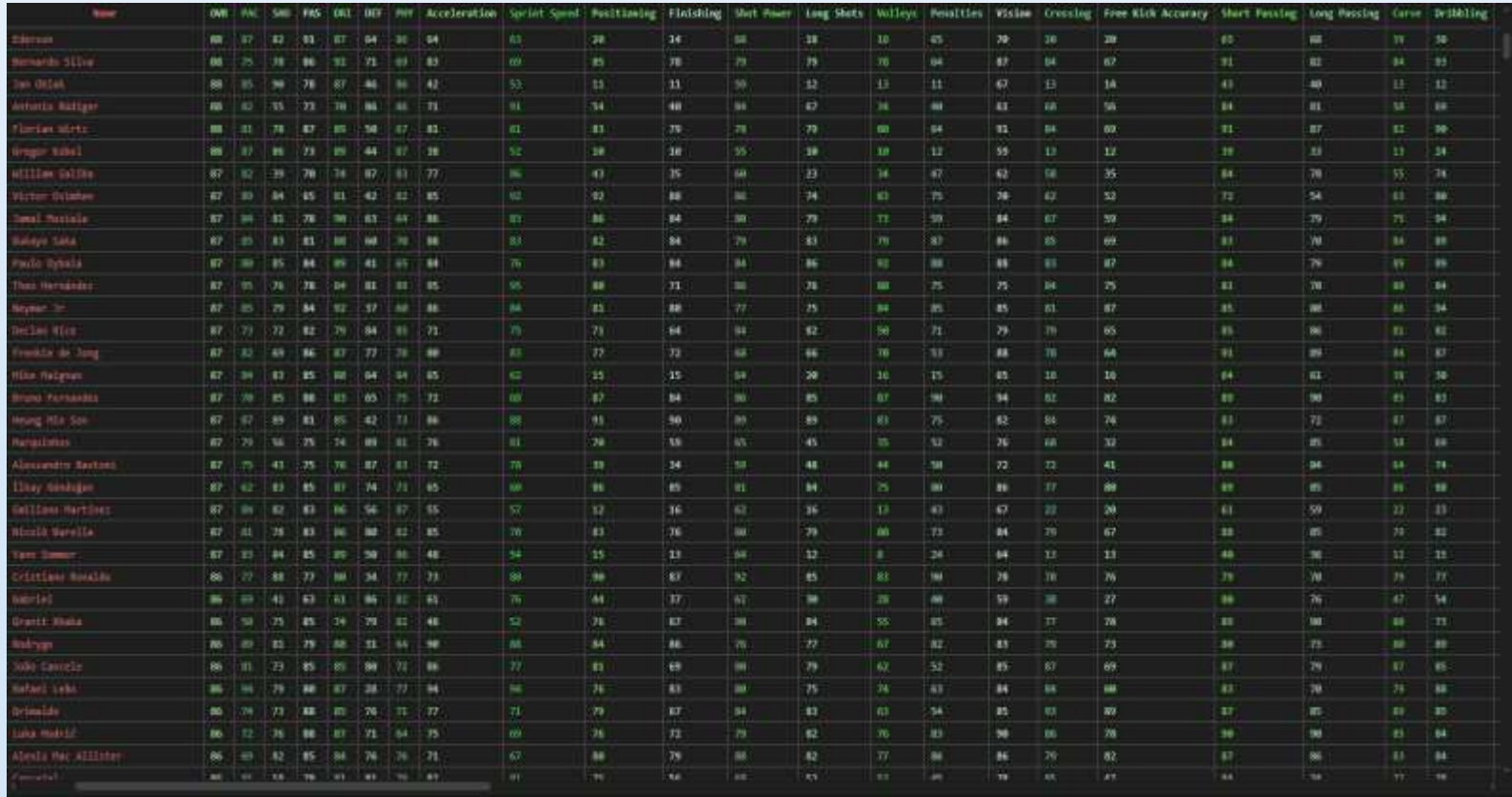
- **La Soluzione:**
- L'architettura MCP collega l'AI ai dati reali.
- Le risposte diventano **deterministiche** (basate su fatti) e non più probabilistiche

I DATI

- **Dominio:** Sports Data Analytics applicata al Calcio.
- **Fonte Dati:** Dataset **EA Sports FC 25** (Kaggle)
- **Dimensione Dataset:**

• **16.161 Osservazioni** (Giocatori analizzati).
• **58 Variabili** (Metriche tecniche e anagrafiche).

```
[16161 rows x 58 columns]
Dimensioni del dataset: (16161, 58)
```



SERVER MCP

Claude analizza l'output del **Server Dati** e seleziona autonomamente i parametri necessa

- Primo server: relativo all’estrazione analisi dei dati tramite
- Librerie Utilizz

```
from mcp.server.fastmcp import FastMCP
import pandas as pd
import os
import sys
from functools import lru_cache
```

- Secondo Server: specializzato esclusivamente nel rendering gra Matplotlib
- Librerie Utilizza

```
from mcp.server.fastmcp import FastMCP
import matplotlib.pyplot as plt
import os
import webbrowser
import time
```

Server Dati

PIPELINE DATI

- **Ingestione Dati:**
- Utilizzo di **pandas.read_csv()** con encoding **'UTF-8'**.
- Recupera la posizione fisica dello script e cost del file gestendo automaticamente le differenze Mac/Linux

- **Data Cleaning (Pipeline):**
- Conversione forzata in numeri delle abilità
- Trasformazione dei dati sporchi in NaN e imputazione a 0.
- Rimozione spazi bianchi dai nomi per migliorare le ricerc
- Ordinamento del DataFrame per OVR decrescente

- **Robustezza e Logging:**
- Gestione eccezioni (**try-except**) per garantire la stabili
- Integrazione di **traceback** per la traccia complet

```python
try:
    #Costruiamo il percorso sicuro per il file
    script_dir = os.path.dirname(__file__)


    player_stats = "data/male_players.csv"


    DATA_FILE = os.path.join(script_dir, player_stats)

    print(f"Caricamento dataset da: {DATA_FILE}", file=sys.stderr)

    df_players = pd.read_csv(DATA_FILE, encoding='UTF-8')

    # Pulizia Dati
    print("Pulizia dati in corso...", file=sys.stderr)

    # Convertiamo le colonne delle abilità in numeriche, gestendo i valori mancanti
    for col in SKILL_COLUMNS:
        if col in df_players.columns:
            df_players[col] = pd.to_numeric(df_players[col], errors='coerce').fillna(0)

    # Puliamo gli spazi bianchi dai nomi dei giocatori
    df_players['Name'] = df_players['Name'].astype(str).str.strip()
    #oridniamo il dataset per OVR decrescente cosi da avere i migliori in cima e semplificare alcune ricerche
    df_players = df_players.sort_values(by='OVR', ascending=False).reset_index(drop=True)
    print("Dataset caricato e pulito. Server pronto.", file=sys.stderr)

except FileNotFoundError:
    print(f"ERRORE: File non trovato: {DATA_FILE}", file=sys.stderr)
    exit()
except Exception as e:
    print(f"ERRORE durante il caricamento: {e}", file=sys.stderr)
    import traceback
    traceback.print_exc(file=sys.stderr)
    exit()
```

Server Dati

FUNZIONE HELPER

IL CODICE

- Principio DRY (*Don't Repeat Yoursel*
- Utilizzo del Decorator lru_cache pe
  le performance in caso di richieste
- Invocata all'interno dei tool senza
  di riscriverla in ognuno di essi
- Applica un filtro booleano
- Il risultato sarà la prima riga convertita
  in un oggetto Pandas Series

```python
#funzione HELPER
@lru_cache(maxsize=128)
def find_player(player_name: str):
    """Cerca un giocatore nel DataFrame ignorando maiuscole/minuscole."""

    filtro = df_players[df_players['Name'].str.contains(player_name, case=False, na=False)]

    if filtro.empty:
        return None

    return filtro.iloc[0]
```

Name	Lionel Messi
OVR	88
PAC	79
SHO	85
PAS	87
DRI	92
DEF	33
PHY	64
Acceleration	86
Sprint Speed	73
Positioning	89
Finishing	85
Shot Power	83
Long Shots	87
Volleys	86
Penalties	75
Vision	90
Crossing	80
Free Kick Accuracy	92
Short Passing	87
Long Passing	86
Curve	93
Dribbling	93
Agility	90

Server Dati

PRIMO TOOL

IL CODICE

```python
### TOOL 1: Profilo Giocatore
@mcp.tool()
@lru_cache(maxsize=128) #salviamo in cache 128 profili giocatori richiesti
def get_player_info(player_name: str) -> str:
    """Recupera le informazioni base di un giocatore."""
    print(f"[Tool 1] Richiesto profilo per: '{player_name}'", file=sys.stderr)

    player_data = find_player(player_name)

    if player_data is None:
        return f"Errore: Giocatore '{player_name}' non trovato."

    #Portiere vs Giocatore di movimento
    if player_data['Position'] == 'GK':
        # Statistiche specifiche per Portieri
        stats = (
            f"PARAMETRI DA PORTIERE:\n"
            f"DIV (Tuffo):    {player_data['GK Diving']} \t REF (Riflessi): {player_data['GK Reflexes']}\n"
            f"HAN (Presa):    {player_data['GK Handling']} \t POS (Piaz.):   {player_data['GK Positioning']}\n"
            f"KIC (Rinvio):   {player_data['GK Kicking']} \t PHY (Fisico):  {player_data['PHY']}\n"
        )
    else:
        # Statistiche standard
        stats = (
            f"PARAMETRI TECNICI:\n"
            f"PAC (Velocità): {player_data['PAC']} \t DRI (Dribbling): {player_data['DRI']}\n"
            f"SHO (Tiro):     {player_data['SHO']} \t DEF (Difesa):    {player_data['DEF']}\n"
            f"PAS (Passaggi): {player_data['PAS']} \t PHY (Fisico):    {player_data['PHY']}\n"
        )
    #risultato
    return (f"--- SCHEDA GIOCATORE: {player_data ['Name']} ---\n"
        f"Club: {player_data['Team']}\n"
        f"Ruolo: {player_data['Position']} | Rating (OVR): {player_data['OVR']}\n"
        f"Età: {player_data['Age']} anni | Altezza: {player_data['Height']}cm | Piede: {player_data['Preferred foot']}\n"
        f"-----------------------------------------\n"
        +stats
    )
```

- Interroga il DataFrame Pandas per estrarre la scheda tecnica del giocatore.
- Gestisce i casi di errore ("Giocatore non trovato") per garantire robustezza.
- Fornisce all'agente i metadati fondamentali (Età, Squadra, Valutazione, ecc..) distinguendo anche del ruolo in campo su cui basare il ragionamento e lo sv

Server Dati

PRIMO TOOL

RISULTATO

S Mi potresti dare il profilo per Giovanni Di Lorenzo? Genera anche il grafico.

Perfetto! Ti recupero le informazioni su Giovanni Di Lorenzo e genero il grafico del suo profilo.

P Get player info

Ecco il profilo completo di **Giovanni Di Lorenzo**!

Informazioni generali:

- **Club**: SSC Napoli
- **Ruolo**: RB (Terzino destro)
- **Rating complessivo (OVR)**: 82
- **Età**: 31 anni
- **Altezza**: 183 cm (6'0")
- **Piede preferito**: Destro

Parametri tecnici:

- **PAC (Velocità)**: 83 - eccellente rapidità per un terzino
- **SHO (Tiro)**: 67 - discreto nel tiro
- **PAS (Passaggi)**: 73 - buona capacità di costruzione
- **DRI (Dribbling)**: 76 - ottimo controllo di palla
- **DEF (Difesa)**: 79 - solida fase difensiva
- **PHY (Fisico)**: 80 - fisicamente robusto

Il Return della funzione viene letto e r
Claude con un linguaggio scorrevole e na

P Get player info

```
--- SCHEDA GIOCATORE: Giovanni Di Lorenzo ---
Club: SSC Napoli
Ruolo: RB | Rating (OVR): 82
Età: 31 anni | Altezza: 183cm / 6'0"cm | Piede: Right
-----------------------------------------
PARAMETRI TECNICI:
PAC (Velocità): 83   DRI (Dribbling): 76
SHO (Tiro):     67   DEF (Difesa):    79
PAS (Passaggi): 73   PHY (Fisico):    80
```

Server Dati

SECONDO TOOL

IL CODICE

l'utente può chiedere *"Quali skill posso analizzare?"*

Il secondo TOOL risponde a questa domanda fornendo all'utente un "Menu" chiaro delle

```python
###TOOL 2: Lista Abilità
@mcp.tool()
@lru_cache(maxsize=1) #La lista è sempre uguale, inutile ricrearla
def list_available_skills() -> str:
    """Restituisce un elenco di tutte le abilità (skill) che possono essere analizzate."""
    print(f"[Tool 2] Richiesta lista abilità", file=sys.stderr)

    formatted_skills = map(lambda skill: f"• {skill}", SKILL_COLUMNS)

    return f"Ecco tutte le abilità disponibili per l'analisi:\n" + "\n".join(formatted_skills)
```

```python
SKILL_COLUMNS = [
    'PAC', 'SHO', 'PAS', 'DRI', 'DEF', 'PHY',
    'Acceleration', 'Sprint Speed', 'Positioning', 'Finishing', 'Shot Power',
    'Long Shots', 'Volleys', 'Penalties', 'Vision', 'Crossing',
    'Free Kick Accuracy', 'Short Passing', 'Long Passing', 'Curve',
    'Dribbling', 'Agility', 'Balance', 'Reactions', 'Ball Control',
    'Composure', 'Interceptions', 'Heading Accuracy', 'Def Awareness',
    'Standing Tackle', 'Sliding Tackle', 'Jumping', 'Stamina', 'Strength',
    'Aggression', 'GK Diving', 'GK Handling', 'GK Kicking', 'GK Positioning', 'GK Reflexes'
]
```

- **Map** per iterare la lista SKILL_COLUMNS
- **Lambda** per aggiungere il bullet point (•) a ogni elemento

SECONDO TOOL

Server Dati

RISULTATO

S Vorrei fare un confronto fra due giocatori ma non conosco le skill disponibili, puoi mostrarle?

Certo! Ti mostro tutte le abilità disponibili per il confronto.

P List available skills

Ecco tutte le abilità disponibili per confrontare i giocatori:

Abilità Generali:

- PAC (Pace - Velocità)
- SHO (Shooting - Tiro)
- PAS (Passing - Passaggio)
- DRI (Dribbling)
- DEF (Defending - Difesa)
- PHY (Physical - Fisico)

Velocità:

- Acceleration (Accelerazione)
- Sprint Speed (Velocità di Sprint)

Tiro:

- Positioning (Posizionamento)
- Finishing (Finalizzazione)
- Shot Power (Potenza di Tiro)
- Long Shots (Tiri da Fuori)
- Volleys (Volée)
- Penalties (Rigori)

Passaggio:

- Vision (Visione di Gioco)

Come posso aiutarti oggi?

Sonnet 4.5

P List available skills

Risposta

Ecco tutte le abilità disponibili per l'analisi:

- PAC
- SHO
- PAS
- DRI
- DEF
- PHY
- Acceleration
- Sprint Speed

Server Dati

TERZO TOOL

IL CODICE

- Esegue un confronto fra due giocatori su una specifica metrica
- Se la skill richiesta non è nella l'esecuzione restituendo un messagg
- Esegue due chiamate separate alla find_player
- I valori numerici vengono estratti **variabili distinte**

```python
### TOOL 3: Confronto Giocatori
@mcp.tool()
@lru_cache(maxsize=32) #utilizziamo 32 slot di cache per confronti ripetuti, alla 33esima volta si ricomincia a cancellare i meno usati
def compare_players(player_a_name: str, player_b_name: str, skill: str) -> str:
    """Confronta due giocatori su una singola abilità (es. 'Finishing' o 'Sprint Speed')."""
    print(f"[Tool 3] Richiesta comparazione: '{player_a_name}' vs '{player_b_name}' su '{skill}'", file=sys.stderr)

    skill_key = None
    for s in SKILL_COLUMNS:
        if s.lower() == skill.lower():
            skill_key = s
            break

    if skill_key is None:
        return f"Errore: Abilità '{skill}' non valida, cerca all'interno della lista abilità quella che preferisci."

    player_a_data = find_player(player_a_name)
    player_b_data = find_player(player_b_name)

    if player_a_data is None:
        return f"Errore: Giocatore '{player_a_name}' non trovato."
    if player_b_data is None:
        return f"Errore: Giocatore '{player_b_name}' non trovato."

    skill_a = player_a_data[skill_key]
    skill_b = player_b_data[skill_key]

    return (f"Comparazione - {skill_key}:\n"
            f"- {player_a_data['Name']}: {skill_a}\n"
            f"- {player_b_data['Name']}: {skill_b}")
```

Server Dati

TERZO TOOL

RISULTATO

Esegue prima un confronto sui **Riflessi** e successiva
offrendo una valutazione chiara delle due abilità

S Vorrei che mi confrontassi Yann Sommer e Alisson nei Riflessi(Reflexes) e nella Presa(Handling)

Perfetto! Ti confronto Yann Sommer e Alisson su queste due abilità da portiere.

P Compare players

P Compare players

Ecco il confronto tra **Yann Sommer** e **Alisson**:

GK Reflexes (Riflessi):

- Yann Sommer: 89
- Alisson: 89
- **Risultato**: Parità perfetta! Entrambi i portieri hanno riflessi eccellenti.

GK Handling (Presa):

- Yann Sommer: 84
- Alisson: 85
- **Risultato**: Alisson è leggermente superiore nella presa della palla, con un punto di vantaggio.

In sintesi, i due portieri sono estremamente simili nelle abilità analizzate. Hanno riflessi identici e prestazioni molto vicine nella gestione del pallone, con un minimo vantaggio per Alisson nella presa.

P Compare players

```
  "player_a_name": "Yann Sommer",
  "player_b_name": "Alisson"
}
```

Risposta

```
Comparazione - GK Reflexes:
- Yann Sommer: 89.0
- Alisson: 89.0
```

P Compare players

P Compare players

```
  "player_a_name": "Yann Sommer",
  "player_b_name": "Alisson"
}
```

Risposta

```
Comparazione - GK Handling:
- Yann Sommer: 84.0
- Alisson: 85.0
```

Server Dati

QUARTO TOOL

IL CODICE

- Trovare il miglior giocatore
  skill scelta dall'utente.
- Possibilità di filtrare in ba
- Ordina le righe del DataFrame
  decrescente in base alla skil
  la prima riga come Series Pan
- Fornisce all'agente risultati
  club e punteggio della skill.

```python
@mcp.tool()
@lru_cache(maxsize=32)
def find_top_player_for_skill(skill: str, min_age: int = 0) -> str:
    """Trova il miglior giocatore per una specifica abilità, sopra una certa età."""
    print(f"[Tool 4] Richiesta top player per: '{skill}' (Età > {min_age})", file=sys.stderr)

    skill_key = None
    for s in SKILL_COLUMNS:
        if s.lower() == skill.lower():
            skill_key = s
            break

    if skill_key is None:
        return f"Errore: Abilità '{skill}' non valida, cerca all'interno della lista abilità quella che preferisci."

    #Filtra per età
    df_filtrato = df_players[df_players['Age'] >= min_age]

    if df_filtrato.empty:
        return f"Nessun giocatore trovato con più di {min_age} anni."

    #Ordina per l'abilità richiesta e prende il migliore
    top_player = df_filtrato.sort_values(by=skill_key, ascending=False).iloc[0]

    return (f"Miglior giocatore per {skill_key} (sopra i {min_age} anni):\n"
            f"- Nome: {top_player['Name']}\n"
            f"- Club: {top_player['Team']}\n"
            f"- Punteggio: {top_player[skill_key]}")
```

Server Dati

QUARTO TOOL

RISULTATO

S Miglior giocatore nel Dribbling con almeno 30 anni

Ti cerco il miglior giocatore nel dribbling con almeno 30 anni.

P Find top player for skill

```
  skill : Dribbling ,
  `min_age`: 30
}
```

Risposta

```
Miglior giocatore per Dribbling (sopra i 30 anni):
- Nome: Neymar Jr
- Club: Al Hilal
- Punteggio: 94
```

Il miglior giocatore nel **Dribbling** con almeno 30 anni è:

Neymar Jr

- **Club**: Al Hilal
- **Punteggio Dribbling**: 94

Neymar rimane uno dei migliori dribblatori al mondo anche dopo i 30 anni, con un punteggio eccezionale di 94 che riflette la sua straordinaria tecnica individuale e capacità di saltare l'uomo.

S Miglior giocatore nella Stamina (Resistenza)

Ti cerco il miglior giocatore nella Stamina.

P Find top player for skill

```
{
  `skill`: `Stamina`
}
```

Risposta

```
Miglior giocatore per Stamina (sopra i 0 anni):
- Nome: Bruno Fernandes
- Club: Man Utd
- Punteggio: 95
```

Il miglior giocatore nella **Stamina (Resistenza)** è:

Bruno Fernandes

- **Club**: Manchester United
- **Punteggio Stamina**: 95

Bruno Fernandes ha una resistenza eccezionale con un punteggio di 95, che gli permette di mantenere un'intensità elevata per tutta la durata della partita. Questa caratteristica è fondamentale per il suo ruolo di centrocampista box-to-box, dove copre enormi distanze sia in fase offensiva che difensiva.

Server Dati

QUINTO TOOL

IL CODICE

- Generare un classifica dei migliori per un ruolo specifico

- **Pipeline di Selezione:**
  - **Filtro:** Maschera booleana sulla co
  - DataFrame gia ordinato decrescente
  - **Slicing:** Selezione dei primi risul

- Utilizziamo una list comprehension

- L'iterazione avviene tramite **.itert**

- concatena i dati di ogni giocatore

```python
### TOOL 5: migliori giocatori per posizione
@mcp.tool()
@lru_cache(maxsize=32) #I ruoli nel calcio sono pochi (PT, DC, CC, ATT...) con maxsize=32 li copri tutti.
def best_player_for_position(position: str, num_players: int = 1) -> str:
    """Restituisce i migliori giocatori per una posizione specifica."""
    player_data = df_players[df_players['Position'].str.contains(position, case=False, na=False)]

    if player_data.empty:
        return None

    #il DataFrame è già ordinato per OVR decrescente quindi prendiamo i primi
    top_players =player_data.head(num_players)

    #LIST COMPREHENSION
    lines = [ f"- {p.Name} (Club: {p.Team}) -> Rating: {p.OVR}" for p in top_players.itertuples()]

    return f"Top {num_players} giocatori per il ruolo '{position}':\n" + "\n".join(lines)

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

```
Pandas(Index=12, _1=12, _2=12, Rank=20, Name='Virgil van Dijk', OVR=89, PAC=78, SHO=60, PAS=71, DRI=71, DEF=89, PHY=86, Acceleration=66, _13=87, Positioning=47, Finishing=52, _16=81, _17=64, Volleys=45, Penalties=62, Vision=67, Crossing=53, _22=70, _23=80, _24=8
6, Curve=60, Dribbling=70, Agility=54, Balance=50, Reactions=88, _30=77, Composure=90, Interceptions=88, _33=87, _34=90, _35=91, _36=86, Jumping=89, Stamina=72, Strength=93, Aggression=85, Position='CB', _42=3, _43=2, _44='Right', Height='193cm / 6\'4"', Weight=
'92kg / 203lb', _47=nan, Age=33, Nation='Holland', League='Premier League', Team='Liverpool', _52='Aerial+, Anticipate, Block, Bruiser, Jockey, Long Ball Pass, Power Header', url='https://www.ea.com/games/ea-sports-fc/ratings/player-ratings/virgil-van-dijk/20337
6', _54=0.0, _55=0.0, _56=0.0, _57=0.0, _58=0.0)
```

Server Dati

QUINTO TOOL

RISULTAT

S Miglior giocatore nel ruolo di Centrale Difensivo(CB)

Cercherò il miglior giocatore nel ruolo di Centrale Difensivo (CB).

P Best player for position

```
{
  `position`: `CB`,
  `num_players`: 1
}
```

Risposta

```
Top 1 giocatori per il ruolo 'CB':
- Virgil van Dijk (Club: Liverpool) -> Rating: 89
```

Il miglior giocatore nel ruolo di Centrale Difensivo (CB) è **Virgil van Dijk** del Liverpool, con un rating di 89.

Van Dijk è considerato uno dei difensori più forti al mondo, noto per la sua presenza fisica, leadership difensiva, e capacità di leggere il gioco. Vuoi sapere di più sulle sue statistiche specifiche o confrontarlo con altri difensori centrali?

S Migliori 3 giocatori nel ruolo di centrocampista centrale (CM)

Cercherò i migliori 3 giocatori nel ruolo di Centrocampista Centrale (CM).

P Best player for position

```
  `num_players`: 3
}
```

Risposta

```
Top 3 giocatori per il ruolo 'CM':
- Kevin De Bruyne (Club: Manchester City) -> Rating: 90
- Martin Ødegaard (Club: Arsenal) -> Rating: 89
- Federico Valverde (Club: Real Madrid) -> Rating: 88
```

I migliori 3 giocatori nel ruolo di Centrocampista Centrale (CM) sono:

1. **Kevin De Bruyne** (Manchester City) - Rating: 90
2. **Martin Ødegaard** (Arsenal) - Rating: 89
3. **Federico Valverde** (Real Madrid) - Rating: 88

De Bruyne guida la classifica come uno dei centrocampisti più completi al mondo, famoso per la sua visione di gioco e precisione nei passaggi. Ødegaard è il capitano dell'Arsenal e un playmaker di grande talento, mentre Valverde si distingue per la sua energia, versatilità e capacità di coprire tutto il campo.

Vuoi approfondire le statistiche specifiche di qualcuno di questi giocatori o confrontarli tra loro?

Server Grafici

PRIMO TOOL

IL CODICE

- strettamente collegato al primo TOOL del server dati
- Estendere le capacità dell'LLM (limitato al testo) generando **visualizzazioni grafiche**
- Creazione di un **Bar Chart Orizzontale** (barh) per le 6 skill principali
- L'immagine viene renderizzata e salvata in una cartella apposita per i grafici in cui si trova
- Utilizzo della libreria **webbrowser** e del protocollo **file://** per lanciare il visualizzatore predefinito dell'utente
- Indica il tempo di apertura del grafico in ms

```python
@mcp.tool()
def plot_profile_chart(player_name: str, skill: list[str],
                       values: list[int], ovr: int) -> str:
    """Genera un grafico a barre orizzontali per il profilo di un singolo giocatore.
       Ideale per visualizzare le skill (PAC, SHO, PAS) ricevute dal server principale."""

    start_time = time.time()

    # Percorso del file
    script_dir = os.path.dirname(__file__)
    folder_path = os.path.join(script_dir,"Grafici")
    os.makedirs(folder_path, exist_ok=True)
    filename = "profilo.png"
    fullpath = os.path.join(folder_path, filename)

    # Configurazione Grafico
    plt.figure(figsize=(8, 6))
    # Colori diversi per ogni barra per renderlo più leggibile
    colors = ['#FF5733', '#28B463', '#F1C40F', '#3498DB', '#8E44AD', '#7F9C8D']
    # Grafico a barre orizzontali
    plt.barh(skill, values, color=colors)

    # Stile
    plt.title(f"Profilo Tecnico: {player_name} (Rating: {ovr})", fontsize=14)
    plt.xlim(0, 100) # Scala fissa 0-100
    plt.grid(axis='x', linestyle='--', alpha=0.3)

    plt.tight_layout()

    # SALVA E APRI
    try:
        # Rimuovi il vecchio file se esiste
        if os.path.exists(fullpath):
            try:
                os.remove(fullpath)
            except:
                pass

        plt.savefig(fullpath)
        plt.close()

        webbrowser.open(f"file://{fullpath}")

        end_time = time.time()
        durata = (end_time - start_time)*1000
        return f"Grafico del profilo di {player_name} generato e aperto in {durata:.0f} ms."
    except Exception as e:
        return f"Errore generazione grafico profilo: {e}"
```

PRIMO TOOL

Server Grafici

RISULTATO

Parametri tecnici:

- **PAC (Velocità)**: 83 - eccellente rapidità per un terzino
- **SHO (Tiro)**: 67 - discreto nel tiro
- **PAS (Passaggi)**: 73 - buona capacità di costruzione
- **DRI (Dribbling)**: 76 - ottimo controllo di palla
- **DEF (Difesa)**: 79 - solida fase difensiva
- **PHY (Fisico)**: 80 - fisicamente robusto

Ora genero il grafico per visualizzare il suo profilo:

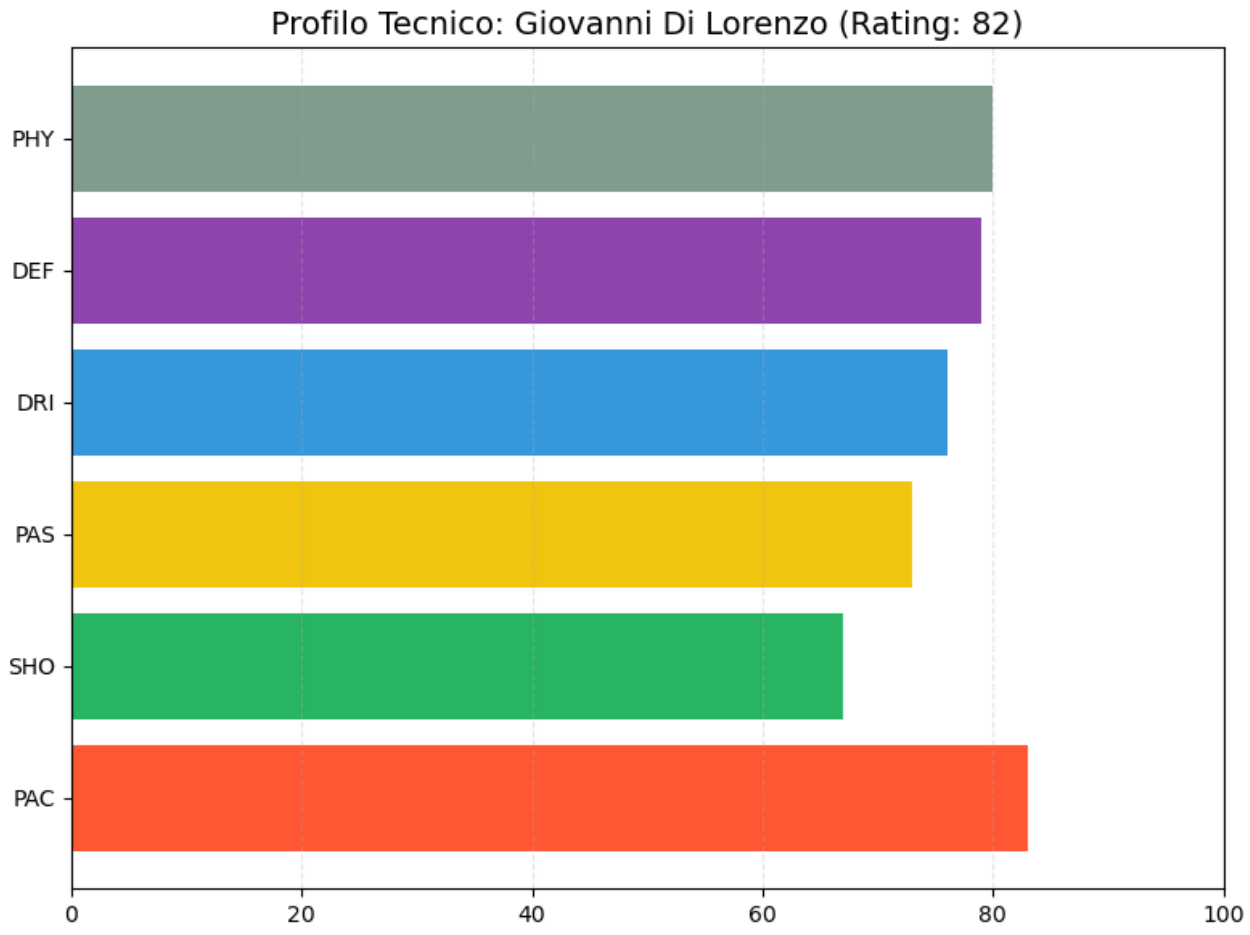
Plot profile chart

```
    79,
    80
  ],
  `player_name`: `Giovanni Di Lorenzo`
}
```

Risposta

```
Grafico del profilo di Giovanni Di Lorenzo generato e aperto in 853 ms.
```

Perfetto! Il grafico del profilo di Giovanni Di Lorenzo è stato generato e dovrebbe essersi aperto nel tuo browser. Il grafico mostra visivamente tutte le sue caratteristiche principali, evidenziando i suoi punti di forza come la velocità (PAC: 83) e il fisico (PHY: 80), tipici di un terzino completo e affidabile.



Server Grafici

SECONDO TOOL

IL CODICE

- I dati del terzo TOOL del server dati possono essere utilizzati Per la creazione di un grafico attraverso l'utilizzo del secondo TOOL del server Grafici

- Creazione del percorso all interno dell'apposita cartella Grafici

- Utilizzo una traslazione simmetrica degli indici delle ascisse. Tramite una **list comprehension** applico un offset di ±width/2 a ogni coordinata del vettore x, questo per separare nettamente le due colonne dei giocatori

- Rimuovo il file se gia esiste e salvo il nuovo

- Apre il file con Webbrowser per lanciare il visualizzatore predefinito dell'utente

- Tempi di apertura del grafico con il modulo Time

```python
### TOOL 2:Grafico Comparazione giocatori
@mcp.tool()
def plot_comparison_chart(skill: list[str], player1_name: str, player1_values: list[int],
                          player2_name: str, player2_values: list[int]) -> str:
    """Genera un grafico comparativo e lo apre nel browser."""
    start_time = time.time()
    # Percorso del file
    script_dir = os.path.dirname(__file__)
    folder_path = os.path.join(script_dir,"Grafici")
    filename = "confronto.png"
    fullpath = os.path.join(folder_path, filename)

    # Configurazione Grafico
    x = range(len(skill))
    width = 0.35
    plt.figure(figsize=(10, 6))

    # Barre affiancate (Blu vs Rosso)
    plt.bar([i - width/2 for i in x], player1_values, width, label=player1_name, color='#3498db')
    plt.bar([i + width/2 for i in x], player2_values, width, label=player2_name, color='#e74c3c')

    # Stile e Decorazioni
    plt.title(f"Confronto Diretto: {player1_name} vs {player2_name}", fontsize=14)
    plt.ylabel('Valore Skill (0-100)')
    plt.ylim(0, 100)
    plt.xticks(x, skill)
    plt.legend()
    plt.grid(axis='y', alpha=0.3)
    plt.tight_layout()

    # SALVA E APRI
    try:
        # Se esiste già, prova a rimuoverlo
        if os.path.exists(fullpath):
            try:
                os.remove(fullpath)
            except:
                pass

        plt.savefig(fullpath)
        plt.close() # Libera la memoria

        webbrowser.open(f"file://{fullpath}")

        end_time = time.time()
        durata = (end_time - start_time)*1000

        return (f"Ho generato il confronto tra {player1_name} e {player2_name}.\n"
                f"Il grafico è stato aperto sul tuo schermo in {durata:.0f} ms.")

    except Exception as e:
        return f"Errore nella generazione del grafico: {e}"
```

SECONDO TOOL

Server Grafici

Ultimo! Creiamo il grafico comparativo:

Plot comparison chart

```
  player2_values : [
    89,
    85
  ]
}
```

Risposta

```
Ho generato il confronto tra Yann Sommer e Alisson.
Il grafico è stato aperto sul tuo schermo in 394 ms.
```

Ecco il confronto tra **Yann Sommer** e **Alisson**:

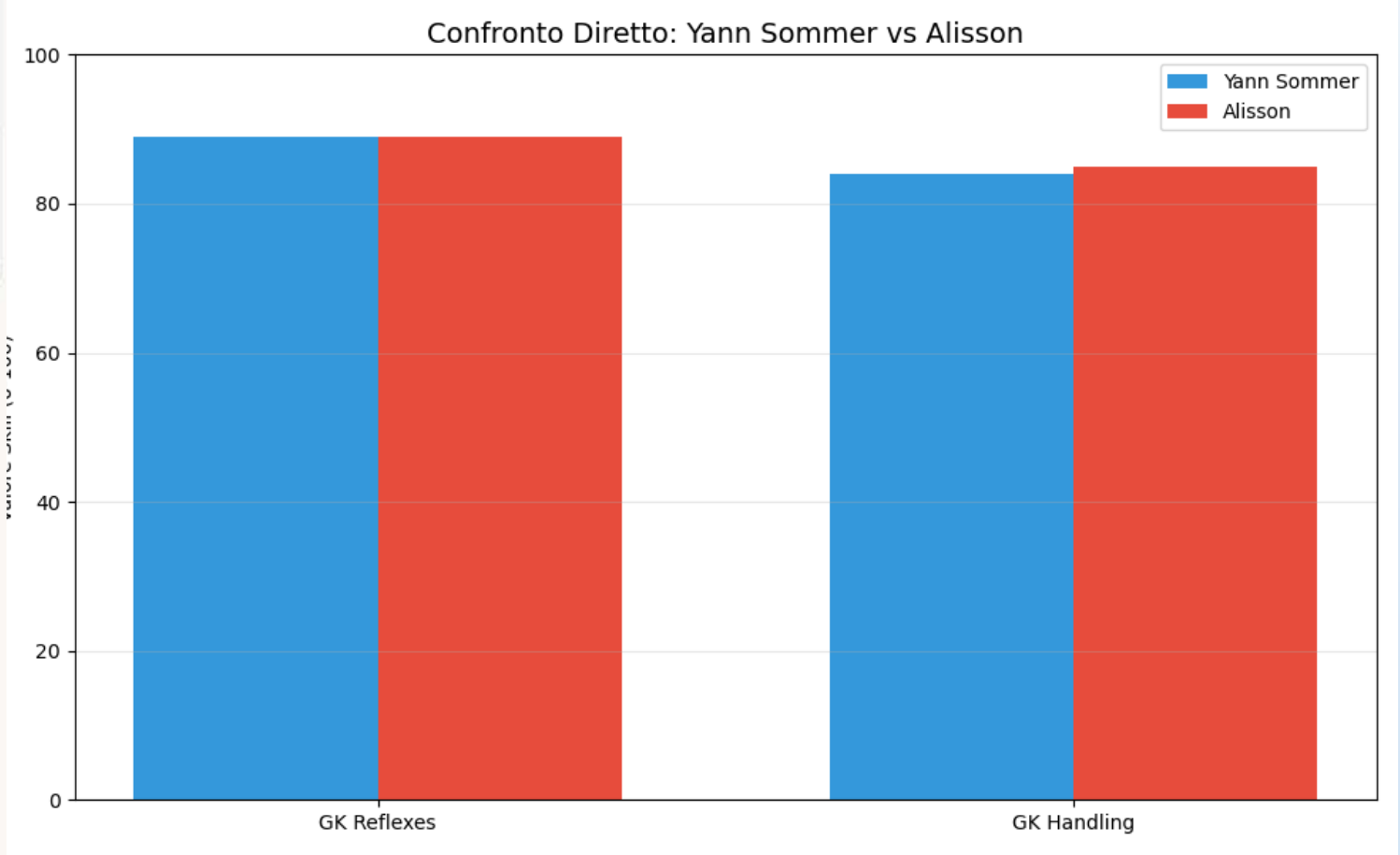
GK Reflexes (Riflessi):

- Yann Sommer: **89**
- Alisson: **89**
- **Parità perfetta** sui riflessi!

GK Handling (Presa):

- Yann Sommer: **84**
- Alisson: **85**
- Alisson ha un leggero vantaggio nella presa

Analisi: I due portieri sono praticamente equivalenti sui riflessi, entrambi con un eccellente valore di 89. Per quanto riguarda la presa del pallone, Alisson si dimostra leggermente superiore con un punto di vantaggio (85 vs 84). Nel complesso, sono due portieri di altissimo livello con caratteristiche molto simili in queste abilità specifiche!

Il grafico comparativo è stato generato e dovrebbe essere visibile sul tuo schermo! 📊

RISULTATO